

7 Modules

Figure 7.1 credit: modification of work "Lone Pine Sunset", by Romain Guy/Flickr, Public Domain

Chapter Outline

7.1 Module basics
7.2 Importing names
7.3 Top-level code
7.4 The help function
7.5 Finding modules
7.6 Chapter summary

Introduction

As programs get longer and more complex, organizing the code into modules is helpful. This chapter shows how to define, import, and find new modules. Python's standard library (https://openstax.org/r/100pythlibrary) provides over 200 built-in modules (https://openstax.org/r/100200modules). Hundreds of thousands of other modules (https://openstax.org/r/100pypi) are available online.

A **module** is a *.py* file containing function definitions and other statements. The module's name is the file's name without the *.py* extension at the end. Ex: The following code, written in a file named *greetings.py*, defines a module named `greetings`.

```
"""Functions that print standard greetings."""

def hello():
  print("Hello!")

def bye():
  print("Goodbye!")
```

Technically, every program in this book is a module. But not every module is designed to run like a program.

Running *greetings.py* as a program would accomplish very little. Two functions would be defined, but the functions would never be called. These functions are intended to be called in other modules.

7.1 Module basics

Learning objectives

By the end of this section you should be able to

- Write a module that consists only of function definitions.
- Import the module and use the functions in a program.

Defining a module

Modules are defined by putting code in a *.py* file. The `area` module below is in a file named area.py. This module provides functions for calculating area.

EXAMPLE 7.1

The area module

```
"""Functions to calculate the area of geometric shapes."""

import math

# 2D shapes

def square(side):
    """Gets the area of a square."""
    return side**2

def rectangle(length, width):
    """Gets the area of a rectangle."""
    return length * width

def triangle(base, height):
    """Gets the area of a triangle."""
    return 0.5 * base * height

def trapezoid(base1, base2, height):
    """Gets the area of a trapezoid."""
    return 0.5 * (base1 + base2) * height

def circle(radius):
    """Gets the area of a circle."""
    return math.pi * radius**2

def ellipse(major, minor):
    """Gets the area of an ellipse."""
```

```
    return math.pi * major * minor

# 3D shapes

def cube(side):
    """Gets the surface area of a cube."""
    return 6 * side**2

def cylinder(radius, height):
    """Gets the surface area of a cylinder."""
    return 2 * math.pi * radius * (radius + height)

def cone(radius, height):
    """Gets the surface area of a cone."""
    return math.pi * radius * (radius + math.hypot(height, radius))

def sphere(radius):
    """Gets the surface area of a sphere."""
    return 4 * math.pi * radius**2
```

CONCEPTS IN PRACTICE

Defining a module

1. How many functions are defined in the `area` module?
   a. 6
   b. 10
   c. 49
2. What would be the result of running *area.py* as a program?
   a. Functions would be defined and ready to be called.
   b. Nothing; the module has no statements to be run.
   c. `SyntaxError`
3. What is the return value of `cube(5)`?
   a. `60`
   b. `125`
   c. `150`

Importing a module

The module defined in *area.py* can be used in other programs. When importing the `area` module, the suffix .py is removed:

```
import area

print("Area of a basketball court:", area.rectangle(94, 50))
```

```
print("Area of a circus ring:", area.circle(21))
```

The output is:

```
Area of a basketball court: 4700
Area of a circus ring: 1385.4423602330987
```

CHECKPOINT

Importing area in a Python shell

Importing area; ch 7, video 1 (https://www.openstax.org/r/importing-area)

CONCEPTS IN PRACTICE

Importing a module

4. What statement would import a module from a file named *secret.py*?
   a. `import secret`
   b. `import secret.py`
   c. `import "secret.py"`
5. What code would return the area of a circle with a radius of 3 meters?
   a. `circle(3)`
   b. `area.circle(3)`
   c. `circle.area(3)`
6. A programmer would like to write a function that calculates the area of a hexagon. Where should the function be written?
   a. the main program
   b. the area module
   c. the secret module

TRY IT

Conversion module

Write a module that defines the following functions:

1. `cel2fah(c)` –
   Converts a temperature in Celsius to Fahrenheit.
   The formula is `9/5 * c + 32`.
2. `fah2cel(f)` –
   Converts a temperature in Fahrenheit to Celsius.
   The formula is `5/9 * (f - 32)`.
3. `km2mi(km)` –
   Converts a distance in kilometers to miles.
   The formula is `km / 1.60934`.

4. `mi2km(mi)` –
   Converts a distance in miles to kilometers.
   The formula is `mi * 1.60934`.

Each function should include a docstring as the first line. A docstring for the module has been provided for you.

The module should not do anything except define functions. When you click the "Run" button, the module should run without error. No output should be displayed.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/7-1-module-basics)

TRY IT

European vacation

Write a program that uses the `conversion` module from the previous exercise to complete a short story. The program's output should match the following example (input in bold):

```
How fast were you driving? 180
Woah, that's like 112 mph!
What was the temperature? 35
That's 95 degrees Fahrenheit!
```

Notice this exercise requires two files:

1. *european.py*, the main program. Input and output statements are provided as a starting point. Edit the lines with TODO comments to use the conversion module.
2. *conversion.py*, the other module. Copy and paste your code from the previous exercise. Import this module in *european.py* after the docstring.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/7-1-module-basics)

7.2 Importing names

Learning objectives

By the end of this section you should be able to

- Import functions from a module using the `from` keyword.
- Explain how to avoid a name collision when importing a module.

The from keyword

Specific functions in a module can be imported using the `from` keyword:

```
from area import triangle, cylinder
```

These functions can be called directly, without referring to the module:

```
print(triangle(1, 2))
print(cylinder(3, 4))
```

EXPLORING FURTHER

As shown below, the `from` keyword can lead to confusing names.

CHECKPOINT

Importing functions

Importing functions; ch 7, video 2 (https://www.openstax.org/r/importing-functions)

CONCEPTS IN PRACTICE

The from keyword

1. Which import statement would be needed before running `print(sqrt(25))`?
   a. `from math import sqrt`
   b. `import sqrt from math`
   c. `import math`
2. How many variables are defined by the statement `from math import sin, cos, tan`?
   a. 0
   b. 3
   c. 4
3. What error would occur if attempting to call a function that was not imported?
   a. `ImportError`
   b. `NameError`
   c. `SyntaxError`

Name collisions

Modules written by different programmers might use the same name for a function. A **name collision** occurs when a function is defined multiple times. If a function is defined more than once, the most recent definition is used:

```
from area import cube

def cube(x):    # Name collision (replaces the imported function)
  return x ** 3

print(cube(2)) # Calls the local cube() function, not area.cube()
```

A programmer might not realize the cube function is defined twice because no error occurs when running the program. Name collisions are not considered errors and often lead to unexpected behavior.

Care should be taken to avoid name collisions. Selecting specific functions from a module to import reduces the memory footprint; however, importing a complete module can help to avoid collisions because a *module.name* format would be used. This is a tradeoff the programmer must consider.

CHECKPOINT

Module and function names

Module and function names; ch 7, video 3 (https://www.openstax.org/r/module-function-names)

CONCEPTS IN PRACTICE

Name collisions

4. A program begins with `from area import square, circle`. What code causes a name collision?
   a.
```
def area(phone):
    """Gets the area code of a phone number."""
```
   b.
```
def circle(x, y, size):
    """Draws a circle centered at (x, y)."""
```
   c.
```
def is_square(length, width):
    """Returns True if length and width are equal."""
```

5. A program begins with `import area`. What code causes a name collision?
   a.
```
area = 51
```
   b.
```
import cylinder from volume
```
   c.
```
def cube(size):
    """Generates a "size X size" rubik's cube."""
```

6. Which line will cause an error?

```
1 def hello():
2   print("Hello!")
3
4 def hello(name):
5   print("Hello,", name)
6
7 hello()
8 hello("Chris")
```

   a. line 4
   b. line 7
   c. line 8

EXPLORING FURTHER

If a name is defined, imported, or assigned multiple times, Python uses the most recent definition. Other languages allow multiple functions to have the same name if the parameters are different. This feature, known as **function overloading**, is not part of the Python language.

TRY IT

Missing imports

Add the missing import statements to the top of the file. Do not make any changes to the rest of the code. In the end, the program should run without errors.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/7-2-importing-names)

TRY IT

Party favors

The following program does not run correctly because of name collisions. Fix the program by modifying import statements, function calls, and variable assignments. The output should be:

```
Bouncy ball area: 13
Bouncy ball volume: 4
Cone hat area: 227
Cone hat volume: 209
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/7-2-importing-names)

7.3 Top-level code

Learning objectives

By the end of this section you should be able to

- Identify code that will run as a side effect of importing.
- Explain the purpose of `if __name__ == "__main__"`.

Side effects

Modules define functions and constants to be used in other programs. When importing a module, *all* code in the module is run from top to bottom. If a module is not designed carefully, unintended code might run as a side effect. The unintended code is generally at the top level, outside of function definitions.

CHECKPOINT

Sphere test code

Sphere test code; ch 7, video 4 (https://www.openstax.org/r/sphere-test)

CONCEPTS IN PRACTICE

Side effects

1. Which line would cause a side effect when imported?

```
1 import math
2
3 print("Defining sphere function")
4
5 def sphere(radius):
6   """Gets the volume of a sphere."""
7   return 4/3 * math.pi * radius**3
```

   a. line 1
   b. line 3
   c. line 5

2. The following *volume.py* module causes a side effect.

```
import math

def sphere(radius):
  """Gets the volume of a sphere."""
  return 4/3 * math.pi * radius**3

for r in range(10000000):
  volume = sphere(r)
```

   a. true
   b. false

3. The following *greeting.py* module causes a side effect.

```
name = input("What is your name? ")
print(f"Nice to meet you, {name}!")
live = input("Where do you live? ")
print(f"{live} is a great place.")
```

   a. true
   b. false

Using __name__

Python modules often include the statement `if __name__ == "__main__"` to prevent side effects. This statement is true when the module is run as a program and false when the module is imported.

CHECKPOINT

The main module

The main module; ch 7, video 5 (https://www.openstax.org/r/main-module)

CONCEPTS IN PRACTICE

Using __name__

4. What is the output when **running** the following *test.py* module?

```
import math

print(math.__name__)
print(__name__)
```

   a. `>math`
      `test`
   b. `__main__`
      `test`
   c. `math`
      `__main__`

5. What is the output when **importing** the following *test.py* module?

```
import math

print(math.__name__)
print(__name__)
```

   a. `math`
      `test`
   b. `__main__`
      `test`
   c. `math`
      `__main__`

6. What line is useful for preventing side effects when importing?
   a. `if __name__ == "main":`
   b. `if __name__ == __main__:`
   c. `if __name__ == "__main__":`

EXPLORING FURTHER

Variables that begin and end with double underscores have special meaning in Python. Double underscores are informally called "dunder" or "magic" variables. Other examples include `__doc__` (the module's docstring) and `__file__` (the module's filename).

TRY IT

Side effects

This exercise is a continuation of the "Missing imports" exercise. Previously, you added missing import

statements to the top of the program. Now, modify the program to prevent side effects when importing the program as a module:

1. Add `if __name__ == "__main__"` at the end.
2. Move all test code under that `if` statement.

The program should run without errors and produce the same output as before.

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/7-3-top-level-code)

TRY IT

Conversion test

This exercise is a continuation of the "Conversion module" exercise. Previously, you wrote the functions `cel2fah`, `fah2cel`, `km2mi`, and `mi2km`. Write test code at the end of *conversion.py* (the original file) for each of these functions. The test code must not run as a side effect when `conversion` is imported by other programs. When running *conversion.py* as the main program, the test output should be:

```
0 C is 32 F
5 C is 41 F
10 C is 50 F
15 C is 59 F
20 C is 68 F

20 F is -7 C
25 F is -4 C
30 F is -1 C
35 F is 2 C
40 F is 4 C

1 km is 0.6 mi
2 km is 1.2 mi
3 km is 1.9 mi
4 km is 2.5 mi
5 km is 3.1 mi

5 mi is 8.0 km
6 mi is 9.7 km
7 mi is 11.3 km
8 mi is 12.9 km
9 mi is 14.5 km
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/7-3-top-level-code)

7.4 The help function

Learning objectives

By the end of this section you should be able to

- Use the `help()` function to explore a module's contents.
- Identify portions of code included in the documentation.

Colors on websites

This section introduces an example module for working with HTML colors (https://openstax.org/r/100htmlcolors). HyperText Markup Language (HTML) is used to design websites and graphical applications. Web browsers like Chrome and Safari read HTML and display the corresponding contents. Ex: The HTML code `<p style="color: Red">Look out!</p>` represents a paragraph with red text.

HTML defines 140 standard color names (https://openstax.org/r/100colornames). Additional colors can be specified using a hexadecimal format: *#RRGGBB*. The digits RR, GG, and BB represent the red, green, and blue components of the color. Ex: *#DC143C* is 220 red + 20 green + 60 blue, which is the color `Crimson`.

Red, green, and blue values range from 0 to 255 (or 00 to FF in hexadecimal). Lower values specify darker colors, and higher values specify lighter colors. Ex: *#008000* is the color `Green`, and *#00FF00* is the color `Lime`.

CHECKPOINT

HTML color codes

HTML color codes; ch 7, video 6 (https://www.openstax.org/r/HTML-color-codes)

CONCEPTS IN PRACTICE

HTML color codes

1. What color is *#000080*?
   a. maroon red
   b. navy blue
   c. olive green
2. What is 255 in hexadecimal?
   a. 00
   b. 80
   c. FF
3. Which color is lighter?
   a. *#FFA500 (orange)*
   b. *#008000 (green)*

Example colors module

A module for working with HTML color codes would be helpful to graphic designers and web developers. The following Python code is in a file named *colors.py*.

- Line 1 is the docstring for the module.

- Lines 3–16 assign variables for frequently used colors.
- Lines 18–24 define a function to be used *within* the module.
- Lines 26–45 define functions to be used in other modules.

Note: The `tohex()` and `torgb()` functions use Python features (string formatting and slicing) described later in the book. For now, the documentation and comments are more important than the implementation details.

```
"""Functions for working with color names and hex/rgb values."""

# Primary colors
RED = "#FF0000"
YELLOW = "#FFFF00"
BLUE = "#0000FF"

# Secondary colors
ORANGE = "#FFA500"
GREEN = "#008000"
VIOLET = "#EE82EE"

# Neutral colors
BLACK = "#000000"
GRAY = "#808080"
WHITE = "#FFFFFF"

def _tohex(value):
  """Converts an integer to an 8-bit (2-digit) hexadecimal string."""
  if value <= 0:
    return "00"
  if value >= 255:
    return "FF"
  return format(value, "02X")

def tohex(r, g, b):
  """Formats red, green, and blue integers as a color in hexadecimal."""
  return "#" + _tohex(r) + _tohex(g) + _tohex(b)

def torgb(color):
  """Converts a color in hexadecimal to red, green, and blue integers."""
  r = int(color[1:3], 16) # First 2 digits
  g = int(color[3:5], 16) # Middle 2 digits
  b = int(color[5:7], 16) # Last 2 digits
  return r, g, b

def lighten(color):
  """Increases the red, green, and blue values of a color by 32 each."""
  r, g, b = torgb(color)
  return tohex(r+32, g+32, b+32)

def darken(color):
```

```
    """Decreases the red, green, and blue values of a color by 32 each."""
    r, g, b = torgb(color)
    return tohex(r-32, g-32, b-32)
```

CONCEPTS IN PRACTICE

The colors module

4. What are the components of the color YELLOW?
   a. red=0, green=255, blue=255
   b. red=255, green=255, blue=0
   c. red=255, green=0, blue=255
5. What code would return a darker shade of blue?
   a. `darken(BLUE)`
   b. `colors.darken(BLUE)`
   c. `colors.darken(colors.BLUE)`
6. What symbol indicates that a function is not intended to be called by other modules?
   a. underscore (_)
   b. number sign (#)
   c. colon (:)

Module documentation

The built-in `help()` function provides a summary of a module's functions and data. Calling `help(module_name)` in a shell is a convenient way to learn about a module.

EXAMPLE 7.2

Output of help(colors) in a shell

The documentation below is automatically generated from the docstrings in *colors.py*.

```
help(colors)

Help on module colors:

NAME
  colors - Functions for working with color names and hex/rgb values.

FUNCTIONS
  darken(color)
    Decreases the red, green, and blue values of a color by 32 each.

  lighten(color)
    Increases the red, green, and blue values of a color by 32 each.

  tohex(r, g, b)
    Formats red, green, and blue integers as a color in hexadecimal.
```

```
  torgb(color)
    Converts a color in hexadecimal to red, green, and blue integers.

DATA
  BLACK = '#000000'
  BLUE = '#0000FF'
  GRAY = '#808080'
  GREEN = '#008000'
  ORANGE = '#FFA500'
  RED = '#FF0000'
  VIOLET = '#EE82EE'
  WHITE = '#FFFFFF'
  YELLOW = '#FFFF00'

FILE
  /home/student/Desktop/colors.py
>>> help(colors)

Help on module colors:

NAME
  colors - Functions for working with color names and hex/rgb values.

FUNCTIONS
  darken(color)
    Decreases the red, green, and blue values of a color by 32 each.

  lighten(color)
    Increases the red, green, and blue values of a color by 32 each.

  tohex(r, g, b)
    Formats red, green, and blue integers as a color in hexadecimal.

  torgb(color)
    Converts a color in hexadecimal to red, green, and blue integers.

DATA
  BLACK = '#000000'
  BLUE = '#0000FF'
  GRAY = '#808080'
  GREEN = '#008000'
  ORANGE = '#FFA500'
  RED = '#FF0000'
  VIOLET = '#EE82EE'
  WHITE = '#FFFFFF'
  YELLOW = '#FFFF00'
```

```
FILE
  /home/student/Desktop/colors.py
```

CONCEPTS IN PRACTICE

The help() function

7. The documentation includes comments from the source code.
   a. true
   b. false
8. In what order are the functions listed in the documentation?
   a. alphabetical order
   b. definition order
   c. random order
9. Which function defined in *colors.py* is not included in the documentation?
   a. `_tohex`
   b. `tohex`
   c. `torgb`

TRY IT

Help on modules

The `random` and `statistics` modules are useful for running scientific experiments. You can become familiar with these two modules by skimming their documentation.

Open a Python shell on your computer, or use the one at python.org/shell (https://openstax.org/r/100pythonshell). Type the following lines, one at a time, into the shell.

- `import random`
- `help(random)`
- `import statistics`
- `help(statistics)`

Many shell environments, including the one on python.org, display the output of `help()` one page at a time. Use the navigation keys on the keyboard (up/down arrows, page up/down, home/end) to read the documentation. When you are finished reading, press the Q key ("quit") to return to the Python shell.

TRY IT

Help on functions

The `help()` function can be called on specific functions in a module. Open a Python shell on your computer, or use the one at python.org/shell (https://openstax.org/r/100pythonshell). Type the following

lines, one at a time, into the shell.

- `import random`
- `help(random.randint)`
- `help(random.choice)`
- `import statistics`
- `help(statistics.median)`
- `help(statistics.mode)`

Remember to use the navigation keys on the keyboard, and press the Q key ("quit") to return to the Python shell.

7.5 Finding modules

Learning objectives

By the end of this section you should be able to

- Explain differences between the standard library and PyPI.
- Search python.org and pypi.org for modules of interest.

Built-in modules

The **Python Standard Library** is a collection of built-in functions and modules that support common programming tasks. Ex: The `math` module provides functions like `sqrt()` and constants like `pi`. Python's official documentation includes a library reference (https://openstax.org/r/100pythlibrary) and a module index (https://openstax.org/r/100200modules) for becoming familiar with the standard library.

For decades, Python has maintained a "batteries included (https://openstax.org/r/100batteries)" philosophy. This philosophy means that the standard library should come with everything most programmers need. In fact, the standard library includes over 200 built-in modules!

Module	Description
`calendar`	General calendar-related functions.
`datetime`	Basic date and time types and functions.
`email`	Generate and process email messages.
`math`	Mathematical functions and constants.
`os`	Interact with the operating system.
`random`	Generate pseudo-random numbers.
`statistics`	Mathematical statistics functions.

Table 7.1 Example built-in modules in the standard library.

Module	Description
`sys`	System-specific parameters and functions.
`turtle`	Educational framework for simple graphics.
`zipfile`	Read and write ZIP-format archive files.

Table 7.1 Example built-in modules in the standard library.

CONCEPTS IN PRACTICE

Built-in modules

Use the library reference, module index, and documentation links above to answer the questions.

1. Which page provides a list of built-in modules sorted by category?
   a. library reference
   b. module index
   c. PEP 2
2. What is the value of `calendar.SUNDAY`?
   a. `1`
   b. `6`
   c. `7`
3. Which built-in module enables the development of graphical user interfaces?
   a. `tkinter`
   b. `turtle`
   c. `webbrowser`

Third-party modules

The **Python Package Index** (PyPI), available at pypi.org (https://openstax.org/r/100pypi), is the official third-party software library for Python. The abbreviation "PyPI" is pronounced like *pie pea eye* (in contrast to PyPy (https://openstax.org/r/100pypy), a different project).

PyPI allows anyone to develop and share modules with the Python community. Module authors include individuals, large companies, and non-profit organizations. PyPI helps programmers install modules and receive updates.

Most software available on PyPI is free and open source. PyPI is supported by the Python Software Foundation (https://openstax.org/r/100foundation) and is maintained by an independent group of developers.

Module	Description
`arrow`	Convert and format dates, times, and timestamps.
`BeautifulSoup`	Extract data from HTML and XML documents.
`bokeh`	Interactive plots and applications in the browser.
`matplotlib`	Static, animated, and interactive visualizations.
`moviepy`	Video editing, compositing, and processing.
`nltk`	Natural language toolkit for human languages.
`numpy`	Fundamental package for numerical computing.
`pandas`	Data analysis, time series, and statistics library.
`pillow`	Image processing for jpg, png, and other formats.
`pytest`	Full-featured testing tool and unit test framework.
`requests`	Elegant HTTP library for connecting to web servers.
`scikit-learn`	Simple, efficient tools for predictive data analysis.
`scipy`	Fundamental algorithms for scientific computing.
`scrapy`	Crawl websites and scrape data from web pages.
`tensorflow`	End-to-end machine learning platform for everyone.

Table 7.2 Example third-party modules available from PyPI.

CONCEPTS IN PRACTICE

Third-party modules

Use pypi.org and the links in the table above to answer the questions.

4. Which modules can be used to edit pictures and videos?
   a. `BeautifulSoup` and `Scrapy`

b. Bokeh and `Matplotlib`
c. `MoviePy` and `Pillow`

5. Which third-party module is a replacement for the built-in `datetime` module?
   a. `arrow`
   b. `calendar`
   c. `time`
6. Search for the webcolors module on PyPI. What function provided by webcolors looks up the color name for a hex code?
   a. `hex_to_name`
   b. `name_to_hex`
   c. `normalize_hex`

EXPLORING FURTHER

Programming blogs often highlight PyPI modules to demonstrate the usefulness of Python. The following examples provide more background information about the modules listed above.

- Top 20 Python Libraries for Data Science for 2023 (https://openstax.org/r/100simplelearn)
- 24 Best Python Libraries You Should Check in 2022 (https://openstax.org/r/100library2022)
- Most Popular Python Packages in 2021 (https://openstax.org/r/100packages2021)

TRY IT

Happy birthday

Module documentation pages often include examples to help programmers become familiar with the module. For this exercise, refer to the following examples from the `datetime` module documentation:

- Examples of Usage: date (https://openstax.org/r/100dateexamples)
- Examples of Usage: timedelta (https://openstax.org/r/100timedeltaex)

Write a program that creates a `date` object representing your birthday. Then get a `date` object representing today's date (the date the program is run). Calculate the difference between the two dates, and output the results in the following format:

```
Your birth date: 2005-03-14
Today's date is: 2023-06-01

You were born 6653 days ago
(that is 574819200 seconds)

You are about 18 years old
```

Access multimedia content (http://openstax.org/books/introduction-python-programming/pages/7-5-finding-modules)

TRY IT

More exact age

The `datetime` module does not provide a built-in way to display a person's exact age. Ex: The following program calculates an exact age (in years and days) using floor division and modulo. The output is: `You are 15 years and 4 days old.`

```
from datetime import date

birth = date(2005, 3, 14)
today = date(2020, 3, 14) # 15 years later
delta = today - birth

years = delta.days // 365
days = delta.days % 365
print("You are", years, "years and", days, "days old")
```

Notice how leap years are included in the calculation. February 29th occurs four times between `birth` and `today`. Therefore, the user is not only 15 years old, but 15 years and 4 days old.

Many commonly used modules from PyPI, including `arrow`, are installed in the Python shell at python.org/shell (https://openstax.org/r/100pythonshell). Open the Python shell and type the following lines:

```
import arrow
birth = arrow.get(2005, 3, 14)
birth.humanize()
```

Refer to the humanize() (https://openstax.org/r/100humanize) examples from the `arrow` module documentation. In the Python shell, figure out how to display the number of years and days since `birth` using *one line of code*. Then display the number of years, months, and days since `birth`. Finally, use the `print()` function to output the results in this format: `You are 18 years 4 months and 7 days old.`

As time permits, experiment with other functions provided by the `arrow` module.

7.6 Chapter summary

Highlights from this chapter include:

- Programs can be organized into multiple *.py* files (modules). The `import` keyword allows a program to use functions defined in another *.py* file.
- The `from` keyword can be used to import specific functions from a module. However, programs should avoid importing (or defining) multiple functions with the same name.
- Modules often include the line `if __name__ == "__main__"` to prevent code from running as a side effect when the module is imported by other programs.
- When working in a shell, the `help()` function can be used to look up the documentation for a module. The documentation is generated from the docstrings.
- Python comes with over 200 built-in modules and hundreds of thousands of third-party modules.

Programmers can search for modules on docs.python.org (https://openstax.org/r/100docstrings) and pypi.org (https://openstax.org/r/100pypi).

Statement	Description
`import module`	Imports a module for use in another program.
`from module import function`	Imports a specific function from a module.
`if __name__ == "__main__":`	A line of code found at the end of many modules. This statement indicates what code to run if the module is executed as a program (in other words, what code *not* to run if this module is imported by another program).
`help(module_name)`	Shows the documentation for the given module. The documentation includes the module's docstring, followed by a list of functions defined in the module, followed by a list of global variables assigned in the module, followed by the module's file name.
`help(function_name)`	Shows the docstring for the given function.
`date(2023, 2, 14)`	Creates a `date` object representing February 14, 2023. Requires: `from datetime import date`.

Table 7.3 Chapter 7 reference.